Section 9 : Controlling Program Flow

Informations sur le papier

- Titre : Programmation C++
- Auteurs : `Etienne KOA`

Aperçu de la section

- Contrôle du déroulement du programme
  - Séquence
    - Ordonner les déclarations de manière séquentielle
  - Sélection
    - Faire des décisions
  - Itération
    - Boucler ou répéter

Sélection - Prise de décision

- Instruction « Si »
- Instruction `If-else`
- Instructions `If` imbriquées
- Instruction `switch`
- Opérateur conditionnel `?:`

Boucle d'itération

- boucle `for`
- Boucle `for` basée sur une plage
- boucle `while`
- boucle `do-while`
- `continue` et `break`
- "Boucles" infinies
- "Boucles" imbriquées

Déclarations « Si »

```
if (expr)
  statement;
```

Figure 1: Si déclarations

- `Si` l'expression est vraie, alors exécutez l'instruction.
- `Si` l'expression est fausse, ignorez l'instruction

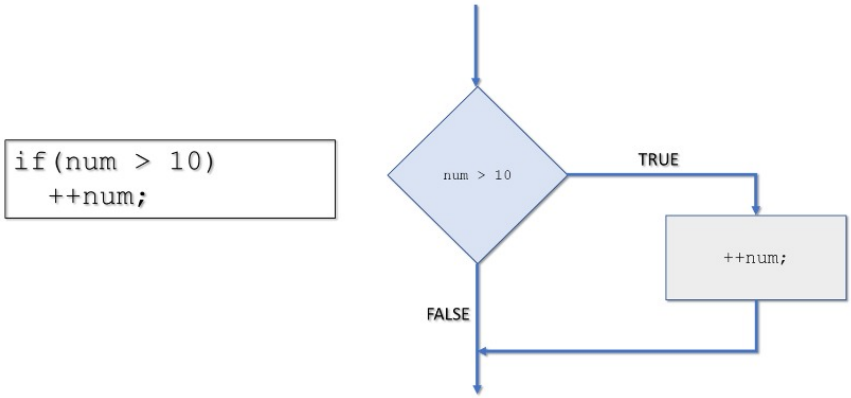

Figure 2: Si déclarations

```
if (selection == 'A')
  cout << "You selected A";

if (num > 10)
  cout << "num is greater than 10";

if (health < 100 && player_healed)
  health = 100;
```

Figure 3: Si déclarations

- instruction de bloc

Instruction `Bloquer`

- Créez un bloc de code en incluant plus d'une instruction dans le code `block{}`.
- Les blocs peuvent également contenir des déclarations de variables.
- Ces variables ne sont visibles que dans la portée locale du bloc.

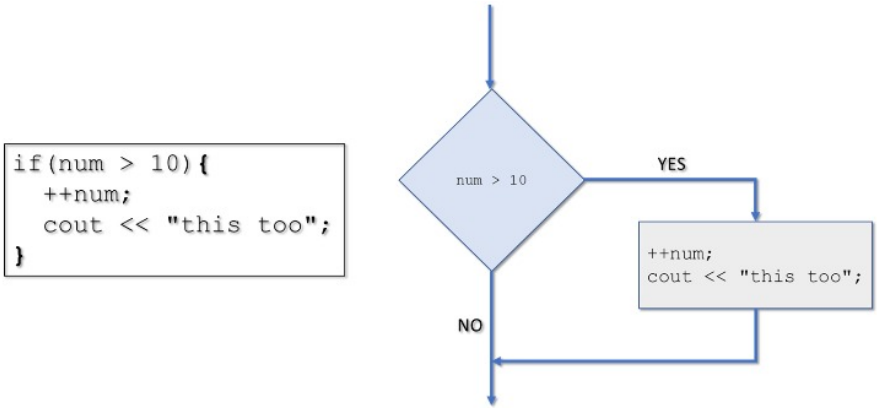

Figure 4: Si déclarations

```
{
    //variable declarations
    statement1;
    statement2;
    . . .
}
```

Figure 5: Instruction de bloc

Déclaration `If-else`

```
if (expr)
    statement1;
else
    statement2;
```

Figure 6: Si-autre

- `Si` l'expression est `true` alors exécutez `statement1`.
- `Si` l'expression est `false` alors exécutez `statement2`.

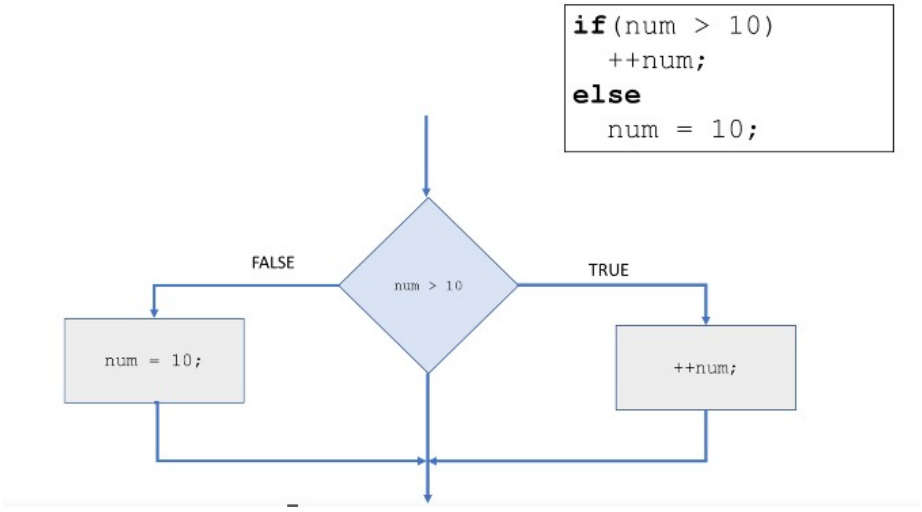

Figure 7: Si-autre

Déclarations `if` imbriquées

- L'instruction `If` est imbriquée dans une autre.
- Permet de tester plusieurs conditions.
- `else` appartient au `if` proche.

```
if (num > 10)
   cout << "num is greater than 10";
else
   cout << "num is NOT greater than 10";


if (health < 100 && heal_player)
   health = 100;
else
   ++health;
```

Figure 8: Si-autre

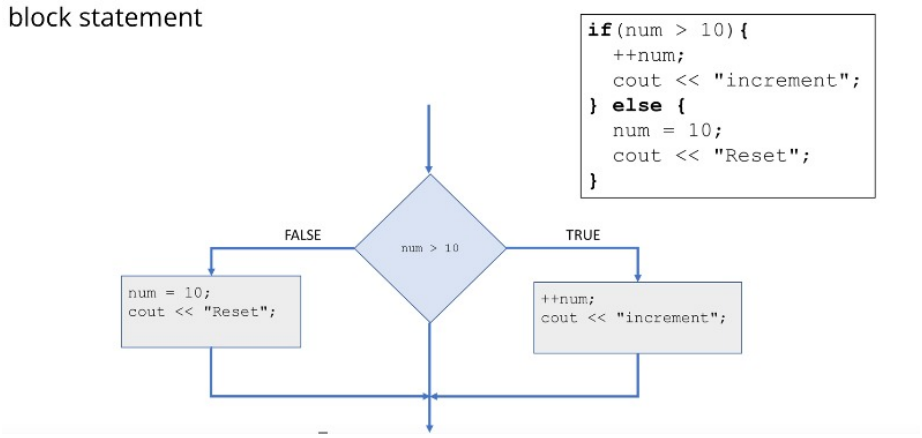

Figure 9: Si-autre

if-else-if construct

block statement

```
if (score > 90)
   cout <<  "A";
else if (score > 80)
   cout <<  "B";
else if (score > 70)
   cout <<  "C";
else if (score > 60)
   cout <<  "D";
else                    // all others must be F
   cout << "F";

cout << "Done";
```

BEGINNING C++ PROGRAMMING
if else Statement

Figure 10: Si-autre

```
if (expr1)
    if (expr2)
        statement1;
    else
        statement2;
```

Figure 11: If

```
if (score > 90)
  if (score > 95)
    cout << "A+";
  else
    cout << "A";

else
  cout << "Sorry, No A";
```

Figure 12: If

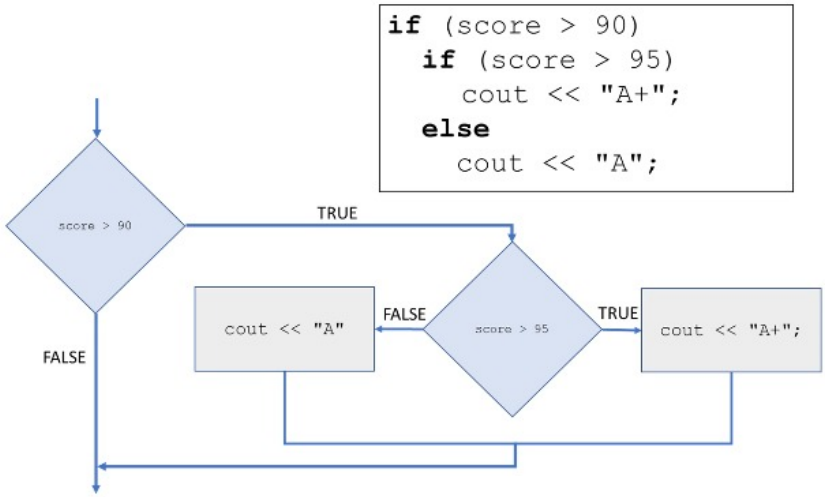

Figure 13: If

```
if (score_frank != score_bill) {
    if (score_frank > score_bill) {
        cout << "Frank Wins" << endl;
    } else {
        cout << "Bill Wins" << endl;
    }
} else {
    cout << "Looks like a tie!" << endl;
}
```

Figure 14: If

```
switch (integer_control_expr) {
   case expression_1: statement_1; break;
   case expression_2: statement_2; break;
   case expression_3: statement_3; break;
     . . .
   case expression_n: statement_n; break;
   default: statement_default;
}
```

Figure 15: Switch

The `switch` statement

- exemple :

```
switch (selection) {
   case '1': cout << "1 selected";
             break;
   case '2': cout << "2 selected";
             break;
   case '3':
   case '4': cout << "3 or 4 selected";
             break;
   default:  cout << "1,2,3,4 NOT selected";
}
```

Figure 16: Switch

- exemple de chute

```
switch (selection) {
   case '1': cout << "1 selected";

   case '2': cout << "2 selected";

   case '3': cout << "3 selected";

   case '4': cout << "4 selected";
             break;
   default:  cout << "1,2,3,4 NOT selected";
}
```

Figure 17: Switch

- avec une énumération
- L'expression de contrôle doit être évaluée comme un type entier.
- L'expression case doit être une expression constante qui s'évalue en entiers ou en littéraux entiers.
- Une fois qu'une correspondance se produit, toutes les sections de cas suivantes sont exécutées « JUSQU'À » ce qu'un « break » soit atteint et que le « switch » soit terminé.

```
enum Color {
    red, green, blue
};
Color screen_color {green};
```

```
switch (screen_color) {
   case red:   cout << "red"; break;
   case green: cout << "green"; break;
   case blue:  cout << "blue"; break;
   default:    cout << "should never execute";
}
```

Figure 18: Switch

- Meilleure pratique : fournissez une instruction de rupture pour chaque cas.
- Meilleure pratique - « par défaut » est facultatif, mais doit être géré.

Opérateur conditionnel

- `?:`

```
(cond_expr) ? expr1 : expr2
```

Figure 19: Opérateur conditionnel

- `cond_expr` est évalué comme une expression booléenne
  - si `cond_expr` est vrai alors la valeur de `expr1` est renvoyée.
  - si `cond_expr` est faux alors la valeur de `expr2` est renvoyée.
- Semblable à la construction `if-else`
- Opérateur ternaire
- Très utile lorsqu'il est utilisé en ligne
- Très facile à abuser !
- Exemple

Boucle

- Itération

```
int a{10}, b{20};
int score{92};
int result {};

result = (a > b) ? a : b;

result = (a < b) ? (b-a) : (a-b);

result = (b != 0) ? (a/b) : 0;

cout << ((score > 90) ? "Excellent" : "Good ");
```

Figure 20: Opérateur conditionnel

  - Le troisième élément de base de la programmation
    - `séquence, sélection, itération`
  - Itération ou répétition
  - Permet l'exécution d'une instruction ou d'un bloc d'instructions de manière répétée.
  - Les boucles sont constituées d'une condition de boucle et du corps qui contient les instructions à répéter.

- Quelques cas d'utilisation typiques
- Exécuter une boucle :
  - un nombre précis de fois
  - pour chaque élément d'une collection
  - tant qu'une condition spécifique reste vraie.
  - jusqu'à ce qu'une condition spécifique devienne fausse.
  - jusqu'à ce que nous atteignions la fin d'un flux d'entrée
  - pour toujours
  - bien d'autres encore
- boucle `for`
  - Répétez un nombre spécifique de fois.
- Boucle `Basée sur une plage`
  - une itération pour chaque élément d'une plage ou d'une collection
- boucle `while`
  - Itérer pendant qu'une condition reste vraie.

  - s'arrêter lorsque la condition devient fausse.
  - vérifier la condition au début de chaque itération.

- boucle `do-while`
  - Itérer pendant qu'une condition reste vraie.
  - Arrêtez-vous lorsqu'une condition devient fausse.
  - vérifier la condition à la fin de chaque itération

```
for (initialization ; condition ; increment)
  statement;


for (initialization ; condition ; increment)  {
  statement(s);
}
```

Figure 21: for Loop

```
int i {0};

for (i = 1 ; i <= 5 ; ++i)
   cout << i << endl;

1
2
3
4
5
```

Figure 22: for Loop

`for` Loop

```
for (int i {1} ; i <= 5 ; ++i)
   cout << i << endl;


for (int i = 1 ; i <= 5 ; ++i)
   cout << i << endl;

i = 100; // ERROR i only visible in the loop
```

Figure 23: for Loop

- array example :

```
int scores [] {100,90,87};

for (int i {0} ; i < 3 ; ++i) {
      cout << scores[i] << endl;
}

for (int i {0} ; i <= 2 ; ++i) {
      cout << scores[i] << endl;
}
100
90
87
```

Figure 24: for Loop

- comma operator
- Quelques autres détails...
  - La boucle for de base est très claire et concise
  - Puisque les expressions de la boucle for sont toutes facultatives, il est possible d'avoir.
    - *pas d'initialisation*

```
for (int i {1}, j {5} ; i <= 5 ; ++i, ++j) {
   cout << i << " * " << j << " : " << (i * j) << endl;
}

1 * 5 : 5
2 * 6 : 12
3 * 7 : 21
4 * 8 : 32
5 * 9 : 45
```

Figure 25: for Loop

* *pas de test*
* *pas d'incrément*

```
for (;;)
   cout << "Endless loop" << endl;
```

Figure 26: for Loop

Range-based `for` Loop

- auto
- vector
- Initializer list
- String

While Loop

- even numbers
- array example
- input validation
- input validation - boolean flag

Introduced in C++11

```
for (var_type var_name: sequence)
   statement; // can use var_name



for (var_type var_name: sequence) {
   statements; // can use var_name
}
```

Figure 27: Range-based for Loop

```
int scores [] {100, 90, 97};

for (int score : scores)
   cout << score << endl;

100
90
97
```

Figure 28: Range-based for Loop

```
int scores [] {100, 90, 97};

for (auto score : scores)
   cout << score << endl;

100
90
97
```

Figure 29: Range-based for Loop

```
vector<double> temps {87.2, 77.1, 80.0, 72.5};

double average_temp {};
double running_sum {};

for (auto temp: temps)
   running_sum += temp;

average_temp = running_sum / temps.size();
```

Figure 30: Range-based for Loop

```
double average_temp {};
double running_sum {};
int size {0};

for (auto temp: {60.2, 80.1, 90.0, 78.2} ) {
   running_sum += temp;
   ++size;
}
average_temp = running_sum / size;
```

Figure 31: Range-based for Loop

```
for (auto c: "Frank")
   cout << c << endl;

F
r
a
n
k
```

Figure 32: Range-based for Loop

```
int i {1};

while (i <= 5) {
   cout << i << endl;
   ++i;  // important!
}

1
2
3
4
5
```

Figure 33: While Loop

```
int i {1};

while (i <= 10) {
   if (i % 2 == 0)
      cout << i << endl;
   ++i;
}

2
4
6
8
10
```

Figure 34: While Loop

```
int scores [] {100,90,87};
int i {0};

while (i < 3){
   cout << scores[i] << endl;
   ++i;
}

100
90
87
```

Figure 35: While Loop

```
int number {};

cout << "Enter an integer less than 100: ";
cin >> number;

while (number >= 100){  // !(number < 100)
  cout << "Enter an integer less than 100";
  cin >> number;
}

cout << "Thanks" << endl;
```

Figure 36: While Loop

```
bool done {false};
int number {0};

while (!done) {
   cout << "Enter an integer between 1 and 5: ";
   cin >> number;
   if (number <=1 || number >=5)
      cout << "Out of range, try again" << endl;
   else {
      cout << "Thanks!" << endl;
      done = true;
   }
}
```

Figure 37: While Loop

```
do {
   statements;
} while (expression);
```

Figure 38: do-while

`do-while` Loop

- Input validation

```
int number {};
do {

   cout << "Enter an integer between 1 and 5: ";
   cin >> number;

} while (number <= 1 || number >= 5);

cout << "Thanks" << endl;
```

Figure 39: do-while

- area calculation with calculate another

```
char selection {};

   do {
      double width {}, height {};
      cout << "Enter width and height separated by a space :";
      cin >> width >> height;

      double area {width * height };
      cout << "The area is " << area << endl;

      cout << "Calculate another? (Y/N) : ";
      cin >> selection;
   } while (selection == 'Y' || selection == 'y');
   cout << "Thanks!" << endl;
```

Figure 40: do-while

Instructions `continue` et `break`

- `continue`
  - Aucune autre instruction dans le corps de la boucle n'est exécutée.
  - Le contrôle passe immédiatement directement au début de la boucle pour l'itération suivante.
- `break`
  - Aucune autre instruction dans le corps de la boucle n'est exécutée.

  - Les boucles sont immédiatement terminées.
  - Control passe immédiatement à l'instruction qui suit la construction de la boucle.

```
vector<int> values {1,2,-1,3,-1,-99,7,8,10};
for (auto val: values) {
  if (val == -99)
     break;
  else if (val == -1)
     continue;
  else
     cout << val << endl;
}
1
2
3
```

Figure 41: continue-break

Boucles infinies

- Boucles dont l'expression de condition est toujours évaluée à vrai.
- Habituellement, cela est involontaire et constitue une erreur du programmeur.
- Parfois, les programmeurs utilisent des boucles infinies et incluent et interrompent des instructions dans le corps pour les contrôler.
- Parfois, des boucles infinies sont exactement ce dont nous avons besoin
  - Boucle d'événements dans un programme événementiel.
  - Système d'exploitation

```
while (true)
   cout << "This will print forever" << endl;
```

Figure 42: infinite-while

- exemple

```
while (true) {
   char again {};
   cout << "Do you want to loop again? (Y/N): ";
   cin >> again;

   if (again == 'N' || again == 'n')
      break;
}
```

Figure 43: infinite-while

Boucles imbriquées

- Boucle imbriquée dans une autre boucle.
- Peut comporter autant de niveaux de profondeur que le programme en a besoin.
- Très utile avec les structures de données multidimensionnelles.
- Boucle externe vs boucle intérieure

```
for (outer_val {1}; outer_val <= 2 ; ++outer_val)
   for (inner_val {1}; inner_val <= 3; ++inner_val)
      cout << outer_val << ", " << inner_val << endl;

1, 1
1, 2
1, 3          outer_val, inner_val
2, 1          Note: inner loop loops "faster"
2, 2
2, 3
```

Figure 44: nested-loops

- Multiplication table:
- 2D Arrays - set all elements to 1000
- 2D Arrays - display elements
- 2D Vrector - display elements

```
for (int num1 {1}; num1 <=10 ; ++num1) {    // outer
   for (int num2 {1}; num2 <=10; ++num2) { // inner
      cout << num1 << " * " << num2
           << " = " << num1 * num2 << endl;
   }
   cout << "-----------" << endl;
}

Displays 10 x 10 Multiplication Table
```

Figure 45: nested-loops

```
int grid[5][3] {};

for (int row {0}; row < 5; ++row ) {
   for (int col {0}; col < 3; ++col ) {
     grid[row][col] = 1000;
   }
}
```

Figure 46: nested-loops

```
for (int row {0}; row < 5; ++row ) {
   for (int col {0}; col < 3; ++col ) {
      cout << grid[row][col] << " ";
   }
   cout << endl;
}
```

Figure 47: nested-loops

```
vector<vector<int>> vector_2d
{
   {1, 2, 3},
   {10, 20, 30, 40},
   {100, 200, 300, 400, 500}
};

for (auto vec: vector_2d) {
   for (auto val: vec) {
      cout << val << " ";
   }
   cout << endl;
}
```

Output

```
1 2 3
10 20 30 40
100 200 300 400 500
```

Figure 48: nested-loops